QuantumNet: Routing and Congestion in a Distributed Quantum Network

Course Project – CSC 4501: Computer Networks

Course	CSC 4501: Computer Networks
Project weight	10% of the course grade
Implementation	Python
Network model	6 QPUs connected by 9 bidirectional links
Prerequisite	Basic Python programming and graph/routing concepts
Prior quantum knowledge	None required

Project objective. This project studies routing and congestion in a simplified distributed quantum network. The quantum setting provides the application context; the technical work remains centered on computer-networking concepts: weighted graphs, shortest paths, reliability, finite link capacity, queueing, congestion, throughput, and controlled experimental comparison.

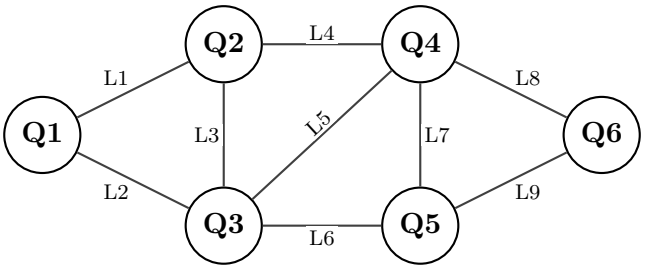
Important. You are not expected to simulate quantum gates, qubits, quantum states, or quantum circuits. Treat an *entanglement request* as an end-to-end connection request between two network nodes.

1. System Model and Minimal Background

A distributed quantum system can be viewed as several quantum processing units (QPUs) connected through communication links. For this project, each QPU is a graph node and each physical link is a graph edge. A request must be routed from a source QPU to a destination QPU through one or more links.

Each link has three properties:

- **Delay** d_e: service time required to use link e.
- **Success probability** p_e: probability that the link-level connection attempt succeeds.
- **Capacity** c_e: maximum number of requests that can simultaneously use the link.



Link	Endpoints	d_e (ms)	p_e	Link	Endpoints	d_e (ms)	p_e
L1	Q1–Q2	8	0.96	L6	Q3–Q5	9	0.91
L2	Q1–Q3	5	0.82	L7	Q4–Q5	4	0.95
L3	Q2–Q3	4	0.88	L8	Q4–Q6	8	0.90
L4	Q2–Q4	7	0.93	L9	Q5–Q6	6	0.94
L5	Q3–Q4	5	0.86				

Figure 1: Project topology. Edge identifiers in the graph correspond to the link parameters shown below. Capacities are $c_e = 2$ for all links except L2, for which $c_{L2} = 3$. The complete topology is provided in `data/topology.json`.

The network contains six QPUs and nine bidirectional links. Multiple source–destination pairs therefore have several feasible routes. For example, a Q1-to-Q6 request can be routed through different intermediate

nodes, and the path with the fewest hops need not have the lowest delay or the highest end-to-end success probability.

2. Provided Repository Structure

Students receive the following files:

```
QuantumNet/
|-- data/
| `-- topology.json
|-- student/
| `-- starter.py
|-- requirements.txt
`-- PROJECT_SPEC.pdf
```

Complete only the numbered `TODO` sections in `starter.py`. Do not rename required functions. The topology file should not be modified for the required experiments.

3. Project Tasks

3.1 Task 1: Network Topology Construction [10 points]

Complete `load_topology(...)`. Your implementation must read the provided JSON file, construct an undirected graph, preserve all edge attributes, verify $0 < p_e \leq 1$, verify $c_e \geq 1$, and verify that the topology is connected.

Required output: a labeled topology figure comparable to Fig. 1.

3.2 Task 2: Traffic Generation [10 points]

Complete `generate_requests(...)`. Requests must arrive according to exponentially distributed inter-arrival times. Each request must contain a request ID, arrival time, source QPU, and destination QPU, with source and destination always different.

You will later vary the arrival rate to represent different levels of offered load.

3.3 Task 3: Routing Policies [30 points]

Implement the following four routing strategies.

(a) Minimum-hop routing. Select the path containing the fewest links.

(b) Minimum-latency routing. For path P,

$$C_{\text{latency}}(P) = \sum_{e \in P} d_e.$$

Select the path with minimum total link delay.

(c) Maximum-reliability routing. Assuming independent link outcomes, the end-to-end success probability of path P is

$$P_{\text{success}}(P) = \prod_{e \in P} p_e.$$

Dijkstra's algorithm requires an additive path cost. Derive an additive link weight that produces the same ordering as maximizing the product above, and implement it in `edge_weight(...)`. Your report must briefly justify the transformation.

(d) Student-designed routing policy. Design one additional metric that uses at least two link properties. You may combine delay, success probability, and/or capacity. State the formula, explain any parameters, and describe the behavior you expect from the metric before discussing the measured results.

3.4 Task 4: Link Capacity and Queueing [15 points]

Implement finite-capacity link service. Each link has c_e simultaneous service slots. If all slots are occupied when a request reaches a link, the request must wait until the earliest slot becomes free.

Use the provided `LinkState` class. A min-heap of slot-availability times is recommended. For each request, track queue waiting time and end-to-end completion time. For each link, track the number of jobs served and utilization.

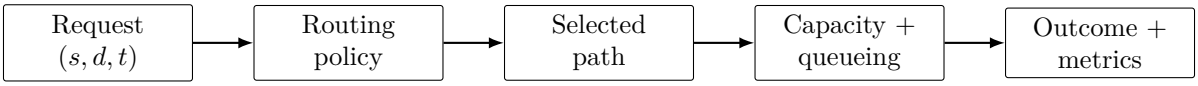


Figure 2: Simulation flow for each generated request. Routing selects a path; finite link capacity may introduce queueing; the simulator then records success/failure and performance metrics.

3.5 Task 5: End-to-End Simulation and Metrics [20 points]

Complete the main simulation loop. For every request:

1. select a route using the chosen policy;
2. process the links along that route in order;
3. reserve link capacity and accumulate any waiting time;
4. sample link success/failure using the provided success probability; and
5. stop after the first failed link or record successful completion.

The simulator must report, at minimum, request success rate, average end-to-end latency, average waiting time, average hop count, throughput, and per-link utilization.

Use the following definitions:

$$\text{Success Rate} = \frac{N_{\text{success}}}{N_{\text{total}}}, \qquad \text{Throughput} = \frac{N_{\text{success}}}{T_{\text{sim}}}.$$

3.6 Task 6: Experimental Evaluation and Technical Report [15 points]

Evaluate all four routing policies under the following offered loads:

$$0.02, \quad 0.05, \quad 0.10, \quad 0.15 \qquad \text{requests/ms.}$$

For a given load, every routing policy must receive the **same request trace**. This controls the workload and makes the comparison fair.

Generate the following four plots:

1. request success rate vs. offered load;
2. average end-to-end latency vs. offered load;
3. throughput vs. offered load; and
4. average queue waiting time vs. offered load.

Your analysis should explain the networking mechanisms behind the results. At minimum, discuss why the three baseline policies can select different paths, how congestion changes their relative behavior, which links become bottlenecks, and when your proposed policy performs well or poorly.

4. Experimental and Reporting Requirements

Use a fixed random seed in the final experiments. A typical run is:

```
python starter.py --topology ../data/topology.json \
                  --output ../outputs/student_run \
                  --requests 500 --seed 42
```

The report should be approximately four pages, excluding references. Organize it as a short technical report rather than a question-by-question worksheet. A recommended structure is:

1. **Introduction and System Model**: problem statement, topology, and link attributes;
2. **Routing Methods**: three baselines and the student-designed policy;
3. **Experimental Setup**: traffic generation, offered loads, metrics, and random seed;
4. **Results and Discussion**: plots, bottlenecks, tradeoffs, and explanation of observed behavior;
5. **Conclusion**: concise summary of the main findings.

5. Submission Checklist

Submit one folder containing:

```
submission/
|-- quantumnet.py
|-- summary.csv
|-- topology.png
|-- success_rate_vs_load.png
|-- latency_vs_load.png
|-- throughput_vs_load.png
|-- waiting_vs_load.png
`-- report.pdf
```

Your code must execute without manual changes after submission. Figures in the report must have axis labels, legends, and captions. Numerical results should be reproducible from the submitted code and fixed seed.

6. Scope and Modeling Assumptions

This project intentionally uses a simplified network model. The topology is static during each experiment, link success events are treated as independent, and no quantum-state simulation is performed. These assumptions keep the project focused on the computer-networking questions of route selection, reliability, capacity, queueing, congestion, and throughput.

Central question: Is the shortest route still the best route when latency, reliability, and congestion must be considered together?